UNIT 2 — Detailed Notes (HTML & Stylesheets/CSS)

PART A: HTML

1. Basics of HTML

- **HTML** = HyperText Markup Language — used to create the structure of web pages.
- It is a **markup language**, not a programming language — uses **tags** to define elements.
- An HTML document is made of **elements**, written using **tags** enclosed in angle brackets `< >`.
- Most tags come in pairs: **opening tag** `<tag>` and **closing tag** `</tag>`. Some are **empty/void tags** (e.g., `<br>`, `<img>`, `<hr>`).

Basic structure of an HTML document:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    Content goes here
  </body>
</html>
```

Important Points: - `<!DOCTYPE html>` declares the document type (HTML5). - `<html>` root element of the page. - `<head>` contains meta-information (title, meta tags, links to CSS/JS) — not displayed directly. - `<body>` contains the visible content of the page.

---

2. Formatting and Fonts

- HTML provides tags to format text appearance.

Common formatting tags: | Tag | Purpose | |—|—| | `<b>` | Bold text | | `<strong>` | Important/bold text (semantic) | | `<i>` | Italic text | | `<em>` | Emphasized/italic text (semantic) | | `<u>` | Underlined text | | `<small>` | Smaller text | | `<mark>` | Highlighted text | | `<sub>` / `<sup>` | Subscript / Superscript | | `<pre>` | Preformatted text (preserves spaces/line breaks) |

Font-related (legacy) tag: - `<font face="" size="" color="">` — deprecated in HTML5; CSS is now used instead.

---

3. Commenting Code

- Comments are notes in the code that are **not displayed** on the webpage — used for explanation/documentation.
- Syntax:

```
<!-- This is a comment -->
```

- Helps in code readability and debugging; ignored by the browser during rendering.

---

4. Color

- Colors can be applied to text, background, borders, etc.
- Ways to specify colors:
  - **Color names** – e.g., `red`, `blue`
  - **HEX codes** – e.g., `#FF0000`
  - **RGB values** – e.g., `rgb(255,0,0)`
  - **RGBA** – includes transparency, e.g., `rgba(255,0,0,0.5)`
  - **HSL** – Hue, Saturation, Lightness

Example:

```
<body bgcolor="#f0f0f0">
<p style="color:blue;">Colored text</p>
```

---

5. Hyperlink

- A **hyperlink** connects one document/resource to another; created using `<a>` (anchor) tag.

Syntax:

```
<a href="destination_url">Link Text</a>
```

Types of links: - **External link** – links to another website. - **Internal link** – links to another page within the same site. - **Anchor link** – links to a section within the same page using `id` and `#`. `html <a href="#section1">Go to Section 1</a> ... <h2 id="section1">Section 1</h2>` - **Email link** – `<a href="mailto:someone@example.com">Email Us</a>` - Attribute `target="_blank"` opens link in a new tab.

---

6. Lists

- **Ordered List** `<ol>` – numbered list.

```
<ol>
  <li>Item 1</li>
  <li>Item 2</li>
</ol>
```

- **Unordered List** `<ul>` – bulleted list.

```
<ul>
  <li>Item A</li>
  <li>Item B</li>
</ul>
```

- **Definition List** `<dl>` – term & description pairs.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
</dl>
```

- Lists can be **nested** (a list inside a list item).

---

7. Tables

- Used to display data in **rows and columns**.

Basic tags: | Tag | Purpose | |—|—| | `<table>` | Defines the table | | `<tr>` | Table row | | `<th>` | Table header cell | | `<td>` | Table data cell | | `<caption>` | Table title/caption |

Example:

```
<table border="1">
  <tr><th>Name</th><th>Marks</th></tr>
  <tr><td>Ravi</td><td>85</td></tr>
</table>
```

Attributes: - `border`, `cellpadding`, `cellspacing`, `colspan`, `rowspan` (for merging cells).

---

8. Images

- Inserted using the `<img>` tag (empty/self-closing tag).

Syntax:

```
<img src="image.jpg" alt="description" width="200" height="150">
```

Important attributes: - `src` – path/URL of the image (required) - `alt` – alternate text (shown if image fails to load; important for accessibility & SEO) - `width`, `height` – dimensions - Images can also act as hyperlinks by wrapping `<img>` inside `<a>`.

---

9. Forms

- Used to **collect user input** and send it to a server for processing.

Basic structure:

```
<form action="submit.php" method="post">
  <input type="text" name="username">
  <input type="submit" value="Submit">
</form>
```

Important attributes: - `action` – URL where form data is sent - `method` – `GET` or `POST`

Common form elements: | Element | Purpose | |—|—| | `<input type="text">` | Single-line text input | | `<input type="password">` | Password field | | `<input type="radio">` | Single choice from options | | `<input type="checkbox">` | Multiple choice selection | | `<input type="submit">` | Submit button | | `<textarea>` | Multi-line text input | | `<select>` / `<option>` | Dropdown list | | `<button>` | Clickable button | | `<label>` | Label for an input field |

---

10. XHTML

- **XHTML** = Extensible HyperText Markup Language — a stricter, XML-based version of HTML.

Key rules of XHTML (differences from HTML): - All tags must be **properly closed** (including empty tags: `<br />`, `<img />`). - Tags must be **properly nested**. - All tag names and attributes must be in **lowercase**. - All attribute values must be **quoted**. - Documents must have a single root element (`<html>`). - Every XHTML document must be **well-formed** (follows XML syntax rules).

---

11. Meta Tags

- `<meta>` tags provide **metadata** about the HTML document (not displayed on page, placed inside `<head>`).

Common uses:

```
        <meta charset="UTF-8">
        <meta name="description" content="Page description">
        <meta name="keywords" content="html, css, web">
        <meta name="viewport" content="width=device-width, initial-
scale=1.0">
        <meta http-equiv="refresh" content="30">
```

- Useful for **SEO**, character encoding, responsive design, and page refresh/redirects.

---

12. Character Entities

- Special characters that have reserved meaning in HTML (like `<`, `>`, `&`) must be represented using **entities**.

Common entities: | Character | Entity Name | Entity Number | |—|—|—| | `<` | `<` | `<` | | `>` | `>` | `>` | | `&` | `&` | `&` | | Space (non-breaking) | ` ` | ` ` | | `"` | `"` | `"` | | © | `©` | `©` |

- Necessary to display these characters correctly without being interpreted as HTML syntax.

---

13. Frames and Frame Sets

- **Frames** allow dividing the browser window into multiple independent sections, each displaying a separate HTML document.
- `<frameset>` replaces `<body>` and defines the layout (rows/columns) of frames.
- `<frame>` defines each individual frame/window.

Example:

```
        <frameset cols="25%,75%">
          <frame src="menu.html">
          <frame src="content.html">
        </frameset>
```

- **Note:** Frames are **deprecated in HTML5** — replaced largely by `<iframe>` (inline frame, embeds a single document within a page) and CSS-based layouts.

Advantages (historical): Reusable navigation menus, independent scrolling of sections. **Disadvantages:** SEO issues, bookmarking problems, accessibility issues — reasons for deprecation.

---

14. Browser Architecture and Website Structure

- (Recap from Unit 1) Browser architecture = UI + Browser Engine + Rendering Engine + Networking + JavaScript Engine + Data Storage.
- **Website structure** refers to how HTML pages are organized and linked together (based on sitemap) — home page, sub-pages, navigation linking them together consistently.

---

15. Overview and Features of the Latest Version of HTML (HTML5)

Key features of HTML5: - **New semantic elements** – `<header>`, `<footer>`, `<article>`, `<section>`, `<nav>`, `<aside>`, `<figure>` (improve structure & readability). - **Multimedia support** – native `<audio>` and `<video>` tags (no plugins like Flash needed). - **`<canvas>`** – for drawing graphics/animations via JavaScript. - **New form input types** – `email`, `date`, `number`, `range`, `url`, `search`, etc. - **Local storage** – `localStorage` and `sessionStorage` for storing data client-side. - **Geolocation API** – to detect user's location. - **Better error handling** – more forgiving parsing rules for malformed markup. - **Drag-and-drop API** support. - **Cross-platform** – designed to work across desktops, tablets, and mobile devices.

PART B: STYLESHEETS (CSS)

16. Need for CSS

- HTML alone only defines **structure**, not appearance — CSS is used to control **presentation** (layout, colors, fonts, spacing).
- Without CSS, all HTML pages would look plain and unstyled.

Advantages of CSS: - **Separation of content and design** (structure in HTML, style in CSS). - **Consistency** – same styles applied across multiple pages. - **Easier maintenance** – change style in one place (CSS file) instead of every page. - **Faster page loading** – less repeated code. - **Responsive design** support (adapting to different screen sizes).

---

17. Introduction to CSS

- **CSS = Cascading Style Sheets.**
- "Cascading" means styles can come from multiple sources, and rules of **priority/specificity** decide which applies.

Ways to include CSS: 1. **Inline CSS** – written directly in an HTML tag using `style` attribute. `html <p style="color:red;">Text</p>` 2. **Internal CSS** – written inside `<style>` tag in the `<head>` section. `html <style> p { color: red; } </style>` 3. **External CSS** – written in a separate `.css` file and linked using `<link>`. `html <link rel="stylesheet" href="style.css">` (Most preferred — best for reusability & maintenance.)

---

18. Basic Syntax and Structure

CSS Rule structure:

```
selector {
  property: value;
}
```

- **Selector** – identifies which HTML element(s) to style.
- **Property** – the style aspect to change (e.g., color, font-size).
- **Value** – the setting for that property.

Example:

```
p {
```

```
  color: blue;
  font-size: 16px;
}
```

Common selector types: | Selector | Example | Meaning | |—|—|—| | Element | `p {}` | All `<p>` tags | | Class | `.classname {}` | Elements with `class="classname"` | | ID | `#idname {}` | Element with `id="idname"` | | Universal | `* {}` | All elements | | Descendant | `div p {}` | `<p>` inside `<div>` |

---

19. Using CSS (Background, Colors and Properties)

- **Background properties:**

```
body {
  background-color: #f0f0f0;
  background-image: url('bg.jpg');
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
}
```

- **Color properties:**

```
h1 {
  color: #333333;   /* text color */
}
```

- Colors can be defined via **names, HEX, RGB, RGBA, HSL** (same as HTML).

---

20. Manipulating Texts

Common text properties:

```
p {
  text-align: center;
  text-decoration: underline;
  text-transform: uppercase;
  line-height: 1.5;
  letter-spacing: 2px;
  word-spacing: 4px;
}
```

- `text-align` – left, right, center, justify
- `text-decoration` – underline, overline, line-through, none
- `text-transform` – uppercase, lowercase, capitalize

---

21. Using Fonts

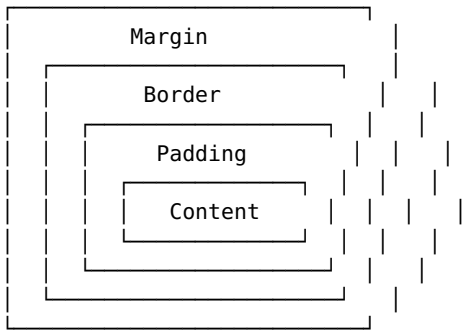
```
p {
  font-family: Arial, sans-serif;
  font-size: 16px;
  font-weight: bold;
  font-style: italic;
}
```

- `font-family` – can list multiple fonts as fallback options.
- **Web fonts** (e.g., Google Fonts) can be imported using `@import` or `<link>` for custom fonts not installed on user's device.

---

22. Borders and Boxes (Box Model)

- Every HTML element is treated as a **box** consisting of:



- **Content** – actual text/image.
- **Padding** – space between content and border.
- **Border** – line surrounding the padding.
- **Margin** – space outside the border, separating from other elements.

Border properties:

```
div {
  border: 2px solid black;
  border-radius: 10px;
}
```

---

23. Margins, Padding, and Lists

Margin & Padding:

```
div {
  margin: 10px 20px 10px 20px; /* top right bottom left */
  padding: 5px;
}
```

- Shorthand order: **top, right, bottom, left** (clockwise).

List styling:

```
ul {
  list-style-type: square;   /* disc, circle, square, none */
  list-style-position: inside;
}
```

---

24. Positioning Using CSS

CSS `position` property values: | Value | Behaviour | |—|—| | `static` | Default; normal document flow | | `relative` | Positioned relative to its normal position | | `absolute` | Positioned relative to nearest positioned ancestor | | `fixed` | Positioned relative to the browser window (stays on scroll) | | `sticky` | Switches between relative & fixed based on scroll position |

Example:

```
.box {
  position: absolute;
  top: 50px;
  left: 100px;
}
```

- Other layout techniques: `float`, `display` (block/inline/inline-block/flex/grid), `z-index` (stacking order).

---

25. CSS2

- **CSS2 (CSS Level 2)** was a major update adding:
  - **Positioning** (absolute, relative, fixed).
  - **Media types** – style sheets for different media (screen, print).
  - **Improved selectors** (attribute selectors, pseudo-classes like `:hover`, `:focus`).
  - **z-index** for stacking order.
  - **Better support for tables and generated content.**
- Built the foundation for later versions (CSS3).

---

26. Overview and Features of the Latest Version of CSS (CSS3)

Key features of CSS3 (modular approach – divided into independent modules): - **Selectors Level 3/4** – more advanced selectors (`:nth-child()`, `:not()`, etc.) - **Box Shadow & Text Shadow** – `box-shadow`, `text-shadow` for visual depth. - **Rounded Corners** – `border-radius`. - **Gradients** – `linear-gradient()`, `radial-gradient()` (no images needed). - **Transitions & Animations** – `transition`, `@keyframes`, `animation` for smooth effects without JavaScript. - **Transformations** – `transform` (rotate, scale, translate, skew). - **Flexbox** – `display: flex` for flexible, responsive one-dimensional layouts. - **Grid Layout** – `display: grid` for two-dimensional layout control. - **Media Queries** – `@media` for **responsive web design** (adapting to screen size/device). - **Custom fonts** – `@font-face` to embed custom web fonts. - **RGBA/HSLA colors** – supports transparency. - **Multiple backgrounds** on a single element.

🗝 Quick Revision Table — Unit 2

Topic	One-line Summary
HTML	Markup language defining webpage structure using tags
Formatting tags	`<b>`, `<i>`, `<u>`, `<strong>`, `<em>`, etc. for text style
Hyperlink	`<a href="">` connects pages/resources
Lists	`<ul>`, `<ol>`, `<dl>` for bulleted/numbered/definition lists
Tables	`<table>`, `<tr>`, `<td>`, `<th>` for rows & columns
Images	`<img src="" alt="">` embeds pictures
Forms	`<form>`, `<input>`, etc. collect user data
XHTML	Stricter, XML-based, well-formed version of HTML
Meta Tags	Metadata in `<head>` (SEO, charset, viewport)
Character Entities	Reserved symbols represented as codes (`<`, `&`)
	Old technique to split browser

Frames	window (deprecated; use `<iframe>`)
HTML5	Semantic tags, audio/video, canvas, storage, geolocation
CSS	Styles the presentation of HTML (colors, fonts, layout)
Box Model	Content + Padding + Border + Margin
Positioning	static, relative, absolute, fixed, sticky
CSS2	Added positioning, media types, pseudo-classes
CSS3	Flexbox, Grid, animations, gradients, media queries, responsive design